P&C Insurance Intelligence System

A multi-agent AI system for P&C (Property & Casualty) insurance claims analysis. Built on the Anthropic Claude API with a Streamlit UI, it ingests insurance PDFs, runs parallel specialist agent analysis, and supports conversational Q&A over a vector-indexed claim corpus.

Table of Contents

System Overview

The system handles three primary workflows:

Workflow	Description	Agents Used
Document Ingestion	Upload a PDF → extract text → LLM classify → embed → store	1 Haiku agent
Claim Summarization	Select a claim → 4 parallel specialists analyse → synthesis → report	4 Haiku + 1 Sonnet
Conversational Chat	Ask questions → orchestrator decides tools → sub-agents execute → streamed answer	1 Opus + up to 2 Haiku sub-agents

The corpus ships pre-loaded: **100 documents across 26 claims** (CLM-00000001 through CLM-00000026), sourced from a Palantir Foundry media set and covering document types including Policy, FNOL, Adjuster Notes, Coverage

Determination, Proof of Loss, Investigation Report, Settlement Agreement, and more.

Architecture Diagram

Technology Stack

Component	Technology	Why
LLM API	Anthropic Claude API (`anthropic>=0.92`)	Native tool use, streaming, structured outputs, adaptive thinking
UI	Streamlit 1.40+	Pure Python, zero JS, rapid iteration, `st.write_stream()` for real-time streaming
Vector Database	ChromaDB 0.5+ (persistent)	Embedded, no server needed, cosine similarity, persists to disk
Embedding Model	`all-MiniLM-L6-v2` (sentence-transformers)	80MB, fast, strong semantic retrieval for English text
PDF Extraction	pdfplumber	Clean text extraction with layout awareness, handles multi-page docs
Async	Python `asyncio`	Parallel specialist agent calls in summarization pipeline
Env Config	python-dotenv	Standard `.env` pattern for API key injection
PDF Generation	reportlab	Used in data prep (`generate_pdfs.py`) — not in the agent pipeline

Database: ChromaDB

Why ChromaDB?

ChromaDB was chosen over alternatives (FAISS, Pinecone, Weaviate, pgvector) for the following reasons:

1. Zero infrastructure overhead

ChromaDB runs embedded in-process using SQLite + HNSW indexes. No Docker, no server, no network calls. The persistent client stores everything in `data/.chroma/` — a regular directory. This matches the project's single-machine, local-first architecture.

2. Pythonic API

ChromaDB's API is collection-oriented and maps cleanly to the insurance domain: a collection holds all document chunks; each chunk has a `document` (text), `metadata` (claim_id, file_type, chunk_index), and is addressable by `id`. Upsert semantics mean re-ingesting the same document is idempotent.

3. Built-in embedding function support

ChromaDB natively wraps `sentence-transformers` via `SentenceTransformerEmbeddingFunction`. You hand it a model name and it handles batched encoding, caching, and device selection — no boilerplate.

4. Metadata filtering

ChromaDB supports `where` clauses on metadata at query time. When a user selects a specific claim in the UI, the search call passes `where={"claim_id": "CLM-00000001"}` to restrict results to that claim's documents only — without a separate filter pass.

5. Scale appropriate for the use case

The corpus is 100 documents / 235 chunks. ChromaDB's HNSW index handles millions of vectors efficiently. For this scale, its overhead is near zero; for 10× or 100× scale it still performs well without requiring a migration to a hosted service.

Schema

```
Collection: "insurance_docs"
Distance metric: cosine

Each chunk document:
  id:        "CLM-00000001__Policy__0"        (claim + file_type + chunk_index)
  document:  "POLICY DOCUMENT\nPolicy Number..."  (raw text chunk, ≤1500 chars)
  metadata:
    claim_id:      "CLM-00000001"
    file_type:     "Policy"
    path:          "data/CLM-00000001/policy_CLM-00000001.pdf"
    chunk_index:   0
    total_chunks:  3
    summary:       "This policy covers..."       (first 500 chars of LLM summary)
```

Chunking Strategy

Documents are split into overlapping chunks of **1500 characters with 200-character overlap**. This was chosen because:

- Insurance documents are typically 1–5 pages of structured text with section headers
- 1500 chars ≈ 300 tokens — fits comfortably in the `n_results` window without hitting context limits
- 200-char overlap ensures clauses that straddle chunk boundaries are captured by both chunks
- Semantic search at the chunk level is more precise than document-level retrieval for targeted Q&A

Agent System Design

1. Ingestion Agent

File: `src/agents/ingestion.py`

Model: `claude-haiku-4-5`

Type: Single synchronous LLM call with structured output

The ingestion pipeline runs when a user uploads a PDF via the sidebar:

```
PDF bytes
    ■
    ▼ pdfplumber
extracted text (all pages concatenated)
    ■
    ▼ claude-haiku-4-5 (structured output / json_schema)
{
  "claim_id": "CLM-...",
  "file_type": "Policy | FNOL | ...",
  "policy_id": "POL-...",
  "insured_name": "...",
  "date_of_loss": "...",
  "claim_amount": "...",
  "cause_of_loss": "..."
}
    ■
    ▼ DocumentStore.add_document()
ChromaDB (chunked + embedded)
```

Why structured output here? The metadata extraction is a strict schema problem — the downstream store needs typed fields. Using `output_config: {format: {type: "json_schema", schema: {...}}}` guarantees valid JSON matching the schema without any parsing logic. Haiku is fast and cheap for this extraction task.

Fallback: If the LLM returns `null` for `claim_id`, the agent falls back to a regex scan of the filename (`CLM-\d+` pattern).

---

2. Summarization Multi-Agent Pipeline

File: `src/agents/summarization.py`

Orchestration: `asyncio.gather` (true parallel execution)

Models: 4× `claude-haiku-4-5` (specialists) + 1× `claude-sonnet-4-6` (synthesis)

This is the core multi-agent pattern: **fan-out to specialists, fan-in to synthesizer**.

Specialist Agents (run in parallel)

All four specialists receive the same document text (up to 6000 chars from all claim documents). Each has a different system prompt and output schema:

Agent	Model	System Prompt Focus	Output Schema

Facts Agent	Haiku	Extract structured factual data	`{claim_id, policy_id, insured_name, date_of_loss, cause_of_loss, total_claimed, adjuster, document_types}`
Coverage Agent	Haiku	Coverage analysis and gap identification	`{coverage_type, policy_period, building_limit, contents_limit, deductible, coverage_status, gaps_identified[]}`
Risk Agent	Haiku	Red flags, anomalies, fraud signals	`{overall_risk: Low/Medium/High, red_flags[], anomalies[], fraud_indicators[], recommendation}`
Timeline Agent	Haiku	Chronological event sequence	`{events: [{date, event, source}], claim_status}`

Each specialist call uses:

- `output_config: {format: {type: "json_schema", schema: ...}}` — guaranteed structured JSON
- `cache_control: {"type": "ephemeral"}` on the system prompt — reduces cost on repeated summarizations of the same claim type

Why `asyncio.gather`?

The four specialists are fully independent — they read the same document text and produce different structured outputs. Running them sequentially would take ~4× longer. With `asyncio.gather`, all four fire simultaneously against the Anthropic API and the total latency is the slowest single call (~1.5s on Haiku) rather than the sum.

```
facts, coverage, risk, timeline = await asyncio.gather(
    self._facts_agent(doc_text),
    self._coverage_agent(doc_text),
    self._risk_agent(doc_text),
    self._timeline_agent(doc_text),
)
```

Synthesis Agent

Model: `claude-sonnet-4-6`

The synthesizer receives all four JSON outputs and produces a human-readable markdown report with sections:

- Executive Summary

- Key Facts
- Coverage Analysis
- Risk Assessment
- Event Timeline
- Recommendations

Sonnet was chosen (rather than Haiku) for synthesis because this step requires:

- Reasoning across four different structured objects
- Identifying connections and contradictions between agent outputs
- Writing professional, coherent prose from structured data

The synthesis system prompt is also cached (`cache_control: {"type": "ephemeral"}`), meaning repeated summarizations across different claims share the cached system context.

---

3. Chat Orchestrator Agent

File: `src/agents/chat.py`

Model: `claude-opus-4-8`

Features: Adaptive thinking, streaming, manual agentic tool loop

The chat agent is the most complex component. It implements a **manual agentic tool loop** — rather than using the SDK's beta tool runner, it drives the loop explicitly for full control over streaming.

Why Manual Loop?

The SDK's `tool_runner` (beta) is convenient but doesn't support token-level streaming to the UI. The manual loop lets us:

- Stream text tokens directly to `st.write_stream()` as they arrive
- Emit tool-call status markers (■ *`Using tool: search_documents...`*) mid-stream
- Accumulate the full `response.content` (including tool_use blocks) for message history

Loop Logic

```
user_message + history
        ■
        ▼
claude-opus-4-8 (adaptive thinking, streaming)
        ■
    stop_reason?
   ■■■■■■■■■■■■■■■■
end_turn      tool_use
   ■              ■
   yield text     execute each tool
   break          yield tool status
                  append tool_results
                  loop back ■■■ claude-opus-4-8
```

Adaptive Thinking

`thinking: {type: "adaptive"}` lets Opus decide per-turn whether to reason deeply before responding. For simple factual lookups ("what is the deductible?") it skips thinking. For complex coverage gap questions it thinks through policy terms, applies them to the claim facts, and reasons about edge cases before answering. This is more token-efficient than always-on extended thinking.

Prompt Caching on the Chat Agent

The system prompt is annotated with `cache_control: {"type": "ephemeral"}`. In a multi-turn conversation, the system prompt is re-sent on every turn (Anthropic's API is stateless). Without caching, a 500-token system prompt costs 500 input tokens × number of turns. With caching, after the first turn it costs ~50 tokens (0.1× the base rate).

---

4. Sub-Agents (Spawned by Tools)

Two tools in the chat agent's toolkit internally spawn their own LLM calls — these are the "sub-agents":

Coverage Sub-Agent (`analyze_coverage` tool)

Model: `claude-haiku-4-5`

Trigger: When the user asks about coverage gaps, deductible application, or coverage disputes

Retrieves policy and claim documents from the store, then calls Haiku with a focused coverage analysis prompt. Returns a structured prose analysis covering:

- Coverage status (covered / partial / excluded)
- Deductible vs. claimed amount comparison
- Coverage gaps or concerns
- Recommendation

Comparison Sub-Agent (`compare_claims` tool)

Model: `claude-haiku-4-5`

Trigger: When the user asks to compare two claims

Retrieves documents for both claims, constructs a side-by-side context window, and calls Haiku to compare claim type, severity, coverage applied, settlement status, and notable differences.

Both sub-agents use `cache_control` on their system prompts for the same cost-reduction reason as the specialists.

Tools Reference

The chat orchestrator has access to 5 tools, defined in `src/tools/retrieval.py`:

Tool	Description	Implementation	Returns

`search_documents`	Semantic search across all 235 chunks	ChromaDB cosine similarity, top-5 results	Formatted text blocks with claim_id, file_type, score, and chunk text
`get_claim_documents`	List all document types for a claim	ChromaDB `get()` with `where` filter, deduplicated by file_type	Comma-separated document type list
`get_claim_summary`	Retrieve cached structured summary	In-memory `summaries_cache` dict lookup	Markdown summary if pre-computed; instruction to use Summary tab if not
`analyze_coverage`	Deep coverage gap analysis	Spawns Haiku sub-agent with policy + claim docs	Prose coverage analysis
`compare_claims`	Side-by-side claim comparison	Spawns Haiku sub-agent with both claims' docs	Structured comparison

Tool description design: Each tool description includes an explicit "when to call this" trigger condition (e.g., *"Call this when answering questions about policy terms, claim details..."*). This is intentional — Opus 4.8 is more conservative about tool use than prior models, and descriptive trigger conditions significantly improve recall of tool invocations for factual questions.

Model Selection Rationale

Model	Where Used	Why
`claude-opus-4-8`	Chat orchestrator	Most capable model for multi-step reasoning, tool selection, and nuanced insurance Q&A. Adaptive thinking handles ambiguous coverage questions. 1M context window accommodates long conversation histories.
`claude-sonnet-4-6`	Summary synthesis	Needs to reason across 4 structured JSON objects and produce coherent professional prose. Sonnet balances quality and speed for this synthesis task — Haiku would produce shallower synthesis; Opus would be overkill.
`claude-haiku-4-5`	All specialists, ingestion, sub-agents	Pure extraction and classification tasks with fixed schemas. Haiku excels at these: fast (~800ms), cheap ($1/$5 per 1M tokens), and accurate on structured output tasks. Running 4 Haiku calls in parallel costs less than 1 Sonnet call and produces comparable structured extraction quality.

Prompt Engineering Decisions

1. Structured Output for All Extraction Tasks

Every specialist agent and the ingestion classifier uses `output_config: {format: {type: "json_schema", schema: {...}}}`. This forces the model to produce valid JSON matching an exact schema — no parsing, no error handling for malformed JSON, no retry loops. The `additionalProperties: false` constraint prevents hallucinated extra fields.

2. Prompt Caching on All System Prompts

Every agent system prompt is marked `cache_control: {"type": "ephemeral"}`. For the chat agent in particular, this reduces input token cost by ~90% on turns 2+ because the 500-token system prompt is served from cache rather than re-processed. For specialist agents, summarizing the same claim type repeatedly (e.g., re-running a summary) also benefits from cache hits.

3. Document Truncation Strategy

The summarization pipeline limits document text to 6000 characters across all documents for specialist agents. This balances completeness against Haiku's context efficiency. For synthesis, the full JSON outputs (typically 200–500 tokens combined) are passed rather than re-truncating the original text.

4. Chat Context Injection

When a user has a specific claim selected in the UI, the chat agent prepends `[Context: User is currently viewing claim CLM-XXXXXXXX]` to the user message. This guides the orchestrator to call `search_documents` or `get_claim_documents` with the correct `claim_id` filter without the user needing to repeat which claim they're asking about.

Data Flow

Pre-loaded Data Path

```
Palantir Foundry (media set)
    ■ (downloaded via download_pdfs.py + Foundry SQL API)
    ▼
data/CLM-XXXXXXXX/*.pdf  (100 PDFs, 1.1MB total)
    ■ (text extracted via generate_pdfs.py → claims_data.json)
    ▼
data/claims_data.json    (100 records: claim_id, file_type, extracted_text, summary)
    ■ (DocumentStore.initialize_from_json() on first app launch)
    ▼
data/.chroma/            (235 chunks, cosine-indexed, persistent)
```

Live Upload Path

```
User uploads PDF via Streamlit sidebar
    ■
    ▼ pdfplumber
Extracted text
    ■
    ▼ claude-haiku-4-5 (json_schema structured output)
{claim_id, file_type, policy_id, insured_name, date_of_loss, ...}
    ■
    ▼ DocumentStore.add_document()
New chunks added to data/.chroma/
```

Summarization Path

```
User clicks "Summarize" for CLM-XXXXXXXX
    ■
    ▼ DocumentStore.get_by_claim()
List of {text, metadata} per document type
    ■
    ▼ asyncio.gather (4 parallel Haiku calls)
facts{...}, coverage{...}, risk{...}, timeline{...}
    ■
    ▼ claude-sonnet-4-6
Markdown report (Executive Summary + 5 sections)
    ■
    ▼ st.session_state.summaries_cache[claim_id]
Cached for this session; also available to chat via get_claim_summary tool
```

Chat Path

```
User types question
    ■
    ▼ claude-opus-4-8 (adaptive thinking, streaming)
Text tokens streamed to UI  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Streamlit chat bubble
    ■
    ▼ (if tool_use)
Tool call: search_documents / analyze_coverage / etc.
    ■
    ▼ execute_tool()
ChromaDB query OR Haiku sub-agent call
    ■
    ▼ tool_result appended to messages
Loop back to claude-opus-4-8
    ■
    ▼ (stop_reason == end_turn)
Final answer streamed to UI
```

Project Structure

```
insurance_agent/
■■■ .env                          # ANTHROPIC_API_KEY (not committed)
■■■ requirements.txt              # All Python dependencies
■■■ claims_data.json              # 100-record extracted text corpus
■■■ generate_pdfs.py              # Synthetic PDF generation (data prep)
■■■ download_pdfs.py              # Foundry media set downloader (data prep)
■
■■■ data/
■   ■■■ claims_data.json          # Extracted text + metadata for all 100 docs
■   ■■■ .chroma/                  # ChromaDB persistent vector store (auto-created)
■   ■■■ CLM-00000001/             # 26 claim folders, each with typed PDFs
■       ■■■ policy_CLM-00000001.pdf
■       ■■■ fnol_CLM-00000001.pdf
■       ■■■ ...
■
■■■ src/
    ■■■ config.py                 # Model names, paths, constants
    ■
    ■■■ tools/
    ■   ■■■ document_store.py     # ChromaDB wrapper: add, search, get_by_claim
    ■   ■■■ pdf_extractor.py      # pdfplumber text extraction + Haiku classifier
    ■   ■■■ retrieval.py          # 5 tool schemas + execute_tool dispatcher
    ■
    ■■■ agents/
    ■   ■■■ ingestion.py          # PDF ingestion pipeline
    ■   ■■■ summarization.py      # Parallel specialist + synthesis orchestrator
    ■   ■■■ chat.py               # Streaming agentic tool loop
    ■
    ■■■ ui/
        ■■■ app.py                # Streamlit: sidebar, summary tab, chat tab
```

Running the System

Prerequisites

```
pip install -r requirements.txt
```

Configuration

```
# Create .env file
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env
```

Launch

```
python3 -m streamlit run src/ui/app.py
```

App starts at `http://localhost:8501`.

On **first launch**, the app automatically loads all 100 documents from `claims_data.json` into ChromaDB (takes ~30 seconds, one-time only — the store persists to `data/.chroma/`).

Using the System

- **Select a claim** from the sidebar dropdown (CLM-00000001 through CLM-00000026)
- **Summary tab** → click "■ Summarize Claim" to run the 5-agent pipeline
    - Watch the status widget show each agent firing
    - Expand "Facts Agent Output", "Coverage Agent Output", etc. to see raw JSON
- **Chat tab** → ask questions in natural language
    - "What is the deductible for this claim?"
    - "Are there any red flags in CLM-00000004?"
    - "Compare CLM-00000001 and CLM-00000002"
    - "Analyze coverage gaps for this claim"
- **Upload tab** (sidebar) → drag a new PDF to ingest it into the corpus

Cost Estimate

A typical session (load app + 1 summarization + 5 chat turns):

Operation	Model	Est. Tokens	Est. Cost
4× specialist agents	Haiku (×4)	~6K in, ~0.5K out each	~$0.008

1× synthesis	Sonnet	~3K in, ~1K out	~$0.024
5× chat turns with tool use	Opus	~5K in, ~1K out each	~$0.15
Total			**~$0.18 per session**

Prompt caching on the chat agent's system prompt reduces the per-turn input cost by ~90% after the first turn.